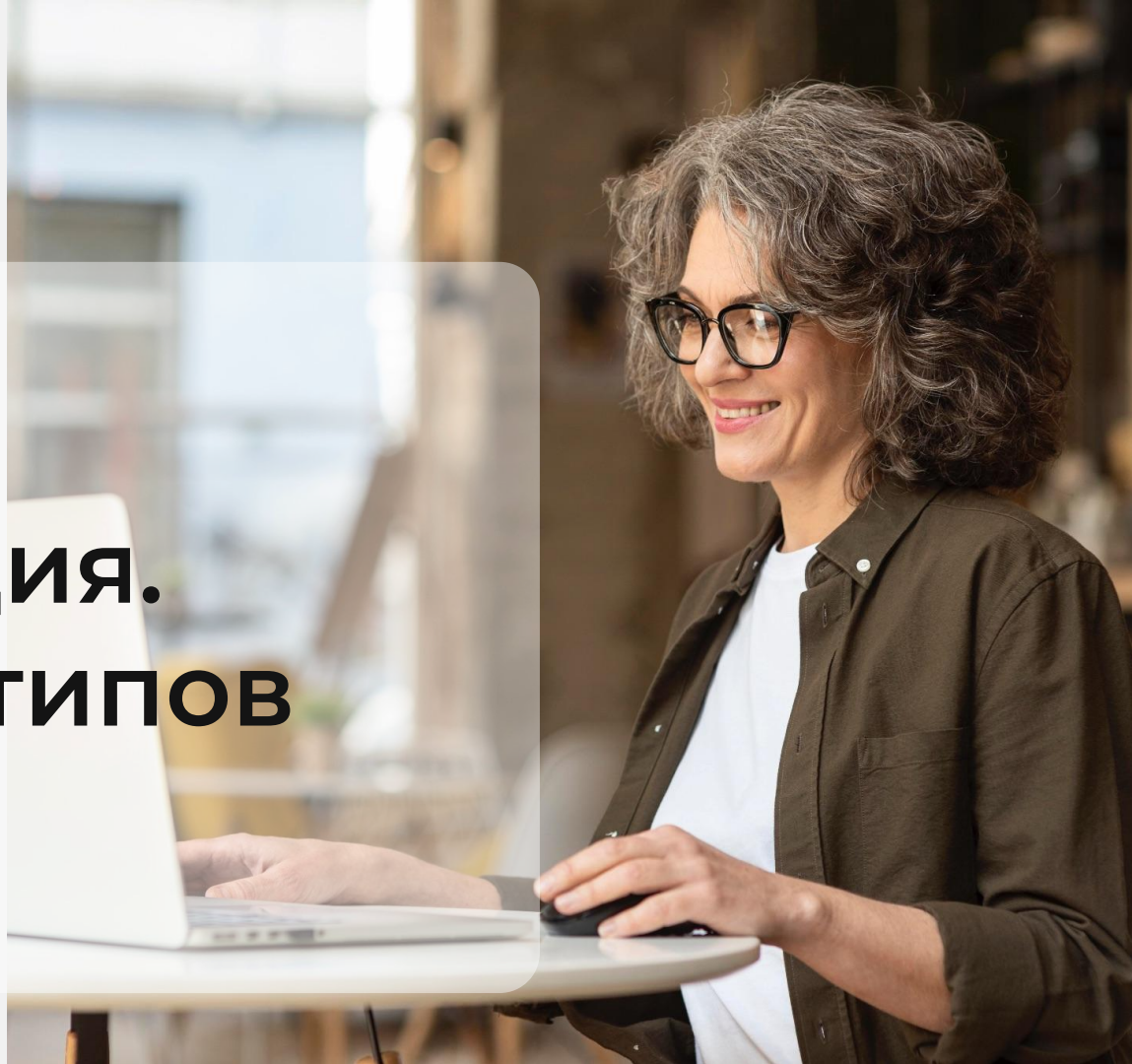


Python

Документация. Аннотации типов



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Передача функций в качестве аргументов
-  Lambda-функции
-  Парадигмы программирования
-  Функциональное программирование
-  Функции высшего порядка
-  Функции map, filter, reduce
-  Функция как ключ в sorted(), min(), max()
-  Параметр key в min() и max()

План занятия

- Документация
- Аннотации типов
- Аннотации для базовых типов
- Аннотации для структур данных
- Аннотации Any, Union и Optional
- Передача неизменяемых и изменяемых объектов



ОСНОВНОЙ БЛОК





Документация



Документация

Это описание кода, которое помогает разработчикам понимать, как работают функции и другие компоненты программы. Она делает код более понятным, облегчает его поддержку и расширение



Docstrings (Документирующие строки)

Это строки документации, заключённые в тройные кавычки (""" или '''), которые используются для описания функций и других компонентов

Docstrings (Документирующие строки)



В отличие от комментариев, `docstrings` являются частью функции и могут быть получены через `help()`

Docstrings



Синтаксис

```
def function_name(param1, param2):
    """
    Описание функции.

    :param param1: Описание первого
    параметра.
    :param param2: Описание второго
    параметра.
    :return: Описание возвращаемого
    значения.
    """
    pass
```

Пример

```
def greet(name):
    """
    Функция принимает имя и возвращает
    строку приветствия.

    :param name: Имя пользователя.
    :return: Приветственное сообщение.
    """
    return f"Hello, {name}!"
```



Функция help

Эта функция используется для просмотра встроенной и пользовательской документации объектов, например функций. Она выводит `docstrings` объекта, если они определены.

Docstrings



Синтаксис

`help(object)`

Пояснения

- `object` — любой объект Python, для которого нужно получить справку.
- Если объект не указан, откроется интерактивный справочный режим

Пример: Получение справки по встроенной функции



```
help(sum)
```

Help on built-in function sum in module builtins:

```
sum(iterable, /, start=0)
```

Return the sum of a 'start' value (default: 0) plus an iterable of numbers

When the iterable is empty, return the start value.

This function is intended specifically for use with numeric values and may reject non-numeric types.

Пример: Просмотр документации пользовательской функции



```
help(greet)
```

```
Help on function greet in module __main__:
```

```
greet(name)
```

```
    Функция принимает имя и возвращает строку приветствия.
```

```
    :param name: Имя пользователя.
```

```
    :return: Приветственное сообщение.
```

Пример: Получение списка методов объекта



```
# Выведет полное описание всех методов строк в Python  
help(str)
```

Функция `help()` также показывает методы и атрибуты объектов.

Вызов справки без аргументов



Если вызвать `help()` без аргумента, откроется интерактивный режим справки

После этого можно вводить имена объектов для получения информации



ВОПРОСЫ





ЗАДАНИЕ



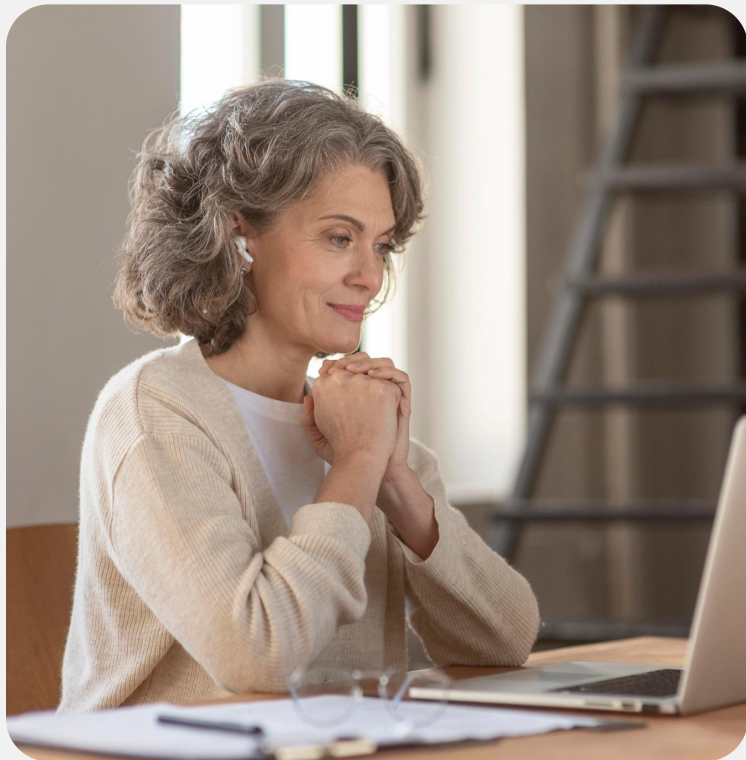


Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
help(print)
```

- a. Выведется документация по функции `print()`
- b. Напечатается инструкция по использованию `help()`
- c. Ошибка, нет скобок у `print`



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
help(print)
```

- a. Выведется документация по функции `print()`
- b. Напечатается инструкция по использованию `help()`
- c. Ошибка, нет скобок у `print`



Ответьте на вопрос

Можно ли использовать `help()` без аргументов?



Ответьте на вопрос

Можно ли использовать `help()` без аргументов?

Да, это откроет интерактивную справку



ВОПРОСЫ





Аннотации типов



Аннотации типов

Это механизм, позволяющий указывать ожидаемые типы аргументов и возвращаемого значения функции, а также типы переменных. Они помогают сделать код понятнее, повысить читаемость и упростить отладку, но не накладывают жёстких ограничений на типы данных

Динамически типизированные языки



Python остаётся **динамически типизированным языком**, и аннотации типов **не заставляют интерпретатор проверять соответствие типов** во время выполнения

Зачем нужны аннотации типов?



Упрощение понимания кода — легче понять, какие данные ожидаются на входе и выходе



Помощь в отладке — статический анализ (проверка кода без выполнения) может обнаружить потенциальные ошибки



Автодокументирование — IDE могут использовать аннотации для подсказок

Аннотации типов



Синтаксис

```
def function_name(param1: type1, param2:
type2, ...) -> return_type:
    # тело функции
    return value

variable: type = value
```

Пояснения

- param1, param2 — имена параметров функции
- type1, type2, type — ожидаемые типы данных для соответствующих параметров или переменных
- -> return_type — аннотация типа возвращаемого значения

Пример аннотации типов



```
def add(a: int, b: int) -> int:
    return a + b
```

```
num: int = 10
```

- `a: int` — аргумент `a` должен быть целым числом
- `b: int` — аргумент `b` должен быть целым числом
- `-> int` — функция должна возвращать целое число
- `num: int` — в переменной можно хранить только целое число

Аннотации не влияют на выполнение кода



```
def add(a: int, b: int) -> int:
    return a + b

print(add(3, 5))
# не приведёт к ошибке, хотя переданы строки
print(add("3", "5"))
```



ВОПРОСЫ





Аннотации для базовых типов

Аннотации для базовых типов



В большинстве случаев название аннотации совпадает с названием типа данных, который она обозначает. Это делает код интуитивно понятным и легко читаемым

Аннотация целых чисел (int)



```
def factorial(n: int) -> int:
    """Возвращает факториал числа."""
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Аннотация чисел с плавающей точкой (float)



```
def convert_to_celsius(fahrenheit: float) -> float:
    """Конвертирует температуру из градусов Фаренгейта в Цельсий."""
    return (fahrenheit - 32) * 5 / 9
```

Аннотация строк (str)



```
def greet(name: str) -> str:  
    """Возвращает приветственное сообщение."""  
    return f"Hello, {name}!"
```

Аннотация логических значений (bool)



```
def is_even(number: int) -> bool:  
    """Определяет, является ли число чётным."""  
    return number % 2 == 0
```

Аннотация None (функция без возврата)



```
def log_message(message: str) -> None:
    """Выводит сообщение в консоль, но не возвращает значения."""
    print(f"LOG: {message}")
```



ВОПРОСЫ





Аннотации для структур данных

Аннотации для структур данных



В Python 3.9+ можно аннотировать списки, кортежи, множества и словари просто указывая встроенные типы. Однако важно понимать, как именно указывать тип содержимого и какие особенности у разных структур

Списки (list)



Синтаксис

`list[element_type]`

Пояснения

- В скобках указывается тип элементов внутри списка
- Рекомендуется указывать тип содержимого

Пример аннотации для списка (list)



```
def process_numbers(numbers: list[int]) -> list[int]:  
    # Список чисел  
    return [n ** 2 for n in numbers]
```

Кортежи (tuple)

Синтаксис



```
tuple[type1, type2]
```

Пояснения

Если кортеж содержит фиксированное число элементов

```
tuple[element_type, ...]
```

Если длина кортежа не фиксирована

Пример аннотации для кортежа (tuple)



```
def get_info() -> tuple[str, float]:
    # Первый элемент str, второй элемент float
    return "Bob", 4.91

def variable_tuple() -> tuple[int, ...]:
    # Кортеж произвольной длины, но только целые числа
    return 5, 8, 2
```

Множества (set)



Синтаксис

`set[element_type]`

Пояснения

- В скобках указывается тип всех элементов множества
- Рекомендуется указывать тип содержимого

Пример аннотации для множества (set)



```
def unique_chars(text: str) -> set[str]:  
    # Множество уникальных символов  
    return set(text)
```


Замороженные множества (frozenset)



Синтаксис

```
frozenset[element_type]
```

Пояснения

- Работает аналогично `set`, но создаёт неизменяемое множество
- Рекомендуется указывать тип содержимого

Пример аннотации для замороженного множества (frozenset)



```
def frozen_example() -> frozenset[int]:  
    # Множество уникальных чисел  
    return frozenset([1, 2, 3])
```

Словари (dict)



Синтаксис

```
dict[key_type, value_type]
```

Пояснения

- В скобках указываются типы ключей и значений
- Рекомендуется указывать типы содержимого

Пример аннотации для словаря (dict)



```
def count_words(text: str) -> dict[str, int]:
    """Принимает строку и возвращает словарь с подсчётом каждого слова."""
    words = text.split()
    # Словарь, где ключи – строки, значения – целые числа
    return {word: words.count(word) for word in words}
```

Коллекции без указания типов



Если тип элементов в коллекции не важен или вы хотите создать коллекцию с элементами разных типов, типы можно не указывать

Пример для коллекции без указания типов



```
# Тип элементов не указан
def process_data(data: list) -> list:
    return [d for d in data if d]

# Элементы должны быть числами
def process_numbers(numbers: list[int]) -> list[int]:
    return [n ** 2 for n in numbers]
```

Коллекции без указания типов



В версиях Python до 3.9 для аннотации структур данных использовались **специальные классы из модуля typing**:

- List вместо list
- Tuple вместо tuple
- Set вместо set
- FrozenSet вместо frozenset
- Dict вместо dict

Аннотация в старых версиях



В Python 3.9+ (рекомендуется)

```
def process_numbers(numbers: list[int]) -> list[int]:
    return [n ** 2 for n in numbers]
```

В Python <3.9 (старый стиль)

```
from typing import List
```

```
def process_numbers_old(numbers: List[int]) -> List[int]:
    return [n ** 2 for n in numbers]
```




ВОПРОСЫ





ЗАДАНИЕ





Исправьте ошибку в коде

```
def greet(name: str) -> str:
    print(f"Hello, {name}!")
```

```
result = greet("Alice")
print(result.upper())
```



Исправьте ошибку в коде

```
def greet(name: str) -> str:
    return f"Hello, {name}!" # Добавлен
return вместо print

result = greet("Alice")
print(result.upper())
```



Ответьте на вопрос



Что произойдёт, если вызвать функцию с неправильным типом аргумента?



Ответьте на вопрос

Что произойдёт, если вызвать функцию с неправильным типом аргумента?

Ответ: Аннотации типов не проверяются во время выполнения, поэтому код выполнится, но может вызвать неожиданное поведение



Ответьте на вопрос

В чём разница между `tuple[str, int]` и `list[str]` в аннотациях?



Ответьте на вопрос

В чём разница между `tuple[str, int]` и `list[str]` в аннотациях?

Ответ: `tuple[str, int]` означает кортеж фиксированной длины, а `list[str]` — список произвольной длины, содержащий строки



ВОПРОСЫ





Аннотации Any, Union и Optional

Аннотации Any, Union и Optional



При аннотировании функций и переменных в Python бывают случаи, когда необходимо указывать **разные возможные типы**. Для этого используются Any, Union и Optional из модуля typing



Any

Эта аннотация означает, что переменная или аргумент могут быть любого типа

Пример для Any



```
from typing import Any

def process_data(data: Any) -> str:
    """Принимает данные любого типа и возвращает строку с их представлением."""
    return f"Данные: {data}"

print(process_data(42))
print(process_data("Hello"))
print(process_data([1, 2, 3]))
```



Union

Эта аннотация позволяет указать, что переменная может содержать один из нескольких типов

Пример для Union



```
from typing import Union

def calculate(value: Union[int, float]) -> float:
    """Принимает число (целое или дробное) и возвращает его квадрат."""
    return value ** 2
print(calculate(5))
print(calculate(2.5))
```



Optional

Аннотация `Optional[T]` — сокращённая запись для `Union[T, None]`, означающая, что значение может быть `None`

Пример для Optional



```
from typing import Optional

def get_user_name(user_id: int) -> Optional[str]:
    """Возвращает имя пользователя или None, если пользователь не найден."""
    users = {1: "Alice", 2: "Bob"}
    return users.get(user_id) # Может вернуть None

print(get_user_name(1))
```

Когда использовать Optional?



Если функция может не возвращать значение



Если переменная может содержать объект или None



Когда значение может отсутствовать (например, поиск в базе данных)



Union с оператором |

В Python 3.10 появился более краткий синтаксис для аннотаций типов — вместо Union можно использовать оператор |

Пример Union с оператором |



```
def calculate(value: int | float) -> float:
    """Принимает число (целое или дробное) и возвращает его квадрат."""
    return value ** 2

print(calculate(5))
print(calculate(2.5))
```



Callable

Эта аннотация используется для аннотирования аргументов или возвращаемого значения, если параметр должен быть функцией

Пример аннотации callable



```
from typing import Callable

def square(x: float) -> float:
    return x * x

def cube(x: float) -> float:
    return x * x * x

def apply_function(func: Callable, value: float) -> float:
    return func(value)

print(apply_function(square, 5))
print(apply_function(cube, 5))
```



ВОПРОСЫ





ЗАДАНИЯ





Ответьте на вопрос

Какой модуль нужен для использования Any, Union и Optional?



Ответьте на вопрос

Какой модуль нужен для использования Any, Union и Optional?

Ответ: typing



Ответьте на вопрос

Какой модуль нужен для использования Any, Union и Optional?



Ответьте на вопрос

Как записать аннотацию, если функция возвращает либо строку, либо None?

Ответ: → `Optional[str]` (или → `Union[str, None]`)



ВОПРОСЫ





Передача неизменяемых и изменяемых объектов

Изменяемые и неизменяемые аргументы функции



Изменяемость передаваемых в функцию объектов влияет на то, будут ли внесённые в аргументы внутри функции изменения затрагивать оригинальные объекты

Передача по значению (копия)

Это передача ссылки на объект, который нельзя изменить. Если объект изменяется внутри функции, создаётся новый объект, а оригинал остаётся неизменным

Применяется к неизменяемым объектам (int, float, bool, str, tuple, frozenset)

Пример передачи по значению (копия, неизменяемый объект)



```
def modify_value(n: int) -> None:
    """Изменяет значение переменной внутри функции (не влияет на оригинал)."""
    print(f"До изменения в функции: {n}, id: {id(n)}")
    n += 1 # Создаётся новый объект
    print(f"После изменения в функции: {n}, id: {id(n)}")

num = 10
modify_value(num)
print(f"Вне функции: {num}, id: {id(num)}")
```

Передача по ссылке (оригинал изменяется)

Это передача ссылки на объект, который можно изменить. Изменение внутри функции затрагивает сам объект

Применяется к изменяемым объектам (list, dict, set)

Пример передачи по ссылке (оригинал изменяется, изменяемый объект)



```
def modify_list(lst: list) -> None:
    """Добавляет элемент в список (изменяет оригинальный объект)."""
    print(f"До изменения в функции: {lst}, id: {id(lst)}")
    lst.append(99) # Изменение оригинального списка
    print(f"После изменения в функции: {lst}, id: {id(lst)}")

my_list = [1, 2, 3]
modify_list(my_list)
print(f"Вне функции: {my_list}, id: {id(my_list)}")
```

Как избежать нежелательных изменений?



Создание копии `сору()` для поверхностного копирования



Использование `деерсору()` для полного копирования вложенных структур

Пример с copy()



```
def safe_modify_list(lst: list) -> list:
    """Работает с копией списка, оставляя оригинальный неизменным."""
    copy_lst = lst.copy()
    copy_lst.append(99)
    return copy_lst

original_list = [1, 2, 3]
new_list = safe_modify_list(original_list)
print(f"Оригинал: {original_list}")
print(f"Копия: {new_list}")
```



ВОПРОСЫ





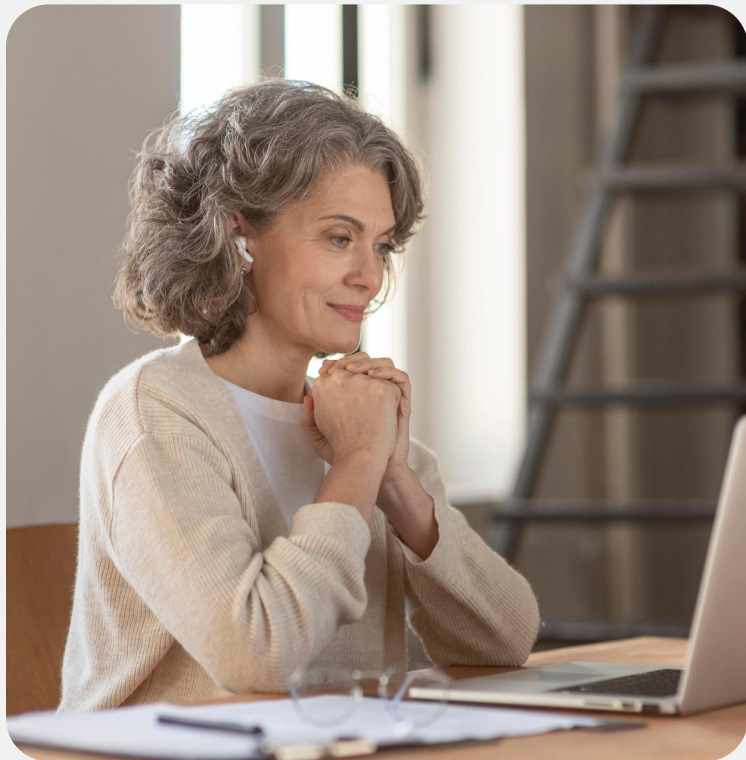
ЗАДАНИЯ





Ответьте на вопрос

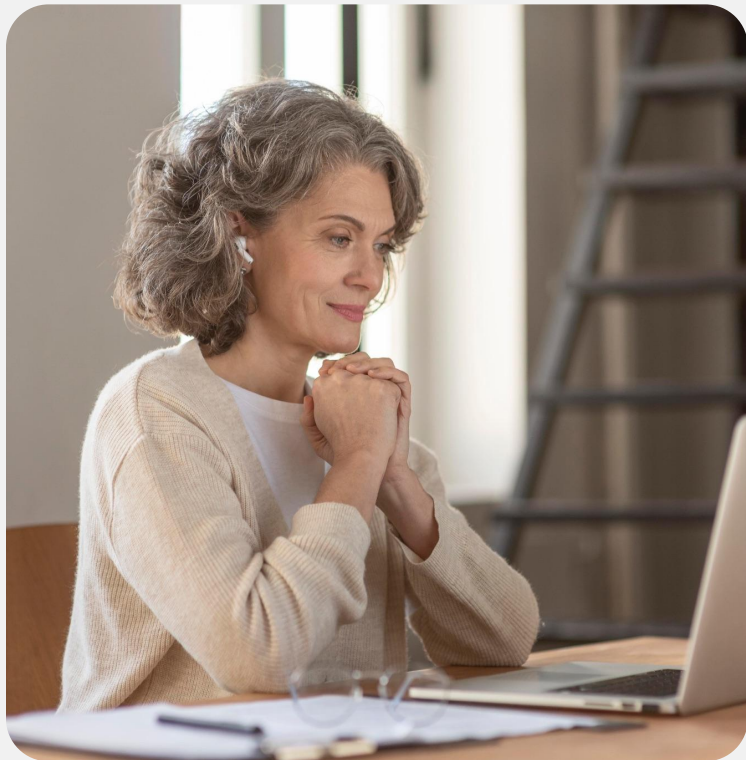
Что произойдёт с переданным списком, если изменить его внутри функции?



Ответьте на вопрос

Что произойдёт с переданным списком, если изменить его внутри функции?

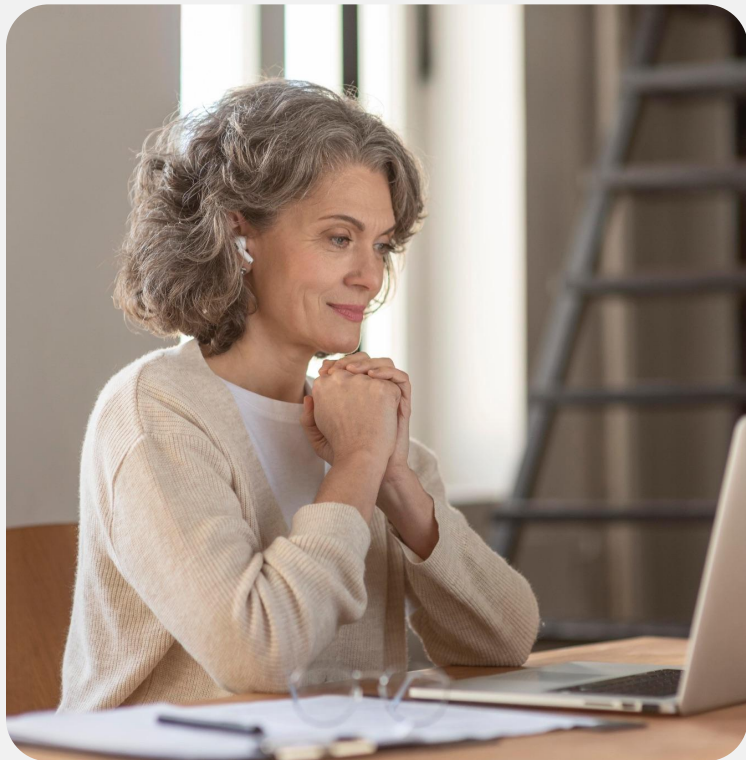
Ответ: **Изменения затронут оригинальный список, так как он передаётся по ссылке**



Ответьте на вопрос



Что делает метод `.sort()` у списка?



Ответьте на вопрос



Что делает метод `.copy()` у списка?

Ответ: Создаёт поверхностную копию списка,
которая не влияет на оригинальный список



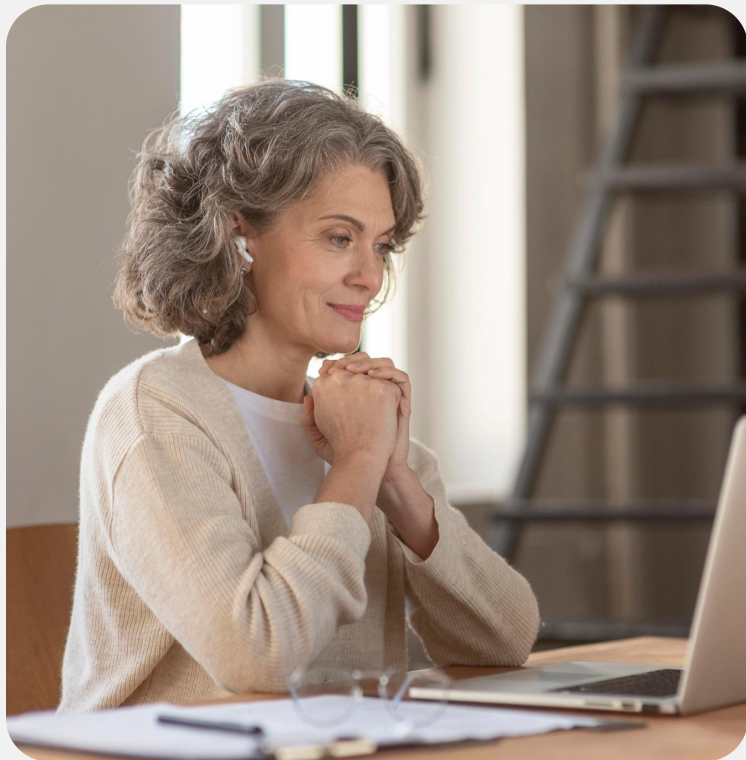
Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def modify_string(text: str) -> str:
    text += "!"
    return text
```

```
original = "Hello"
modify_string(original)
print(original)
```

- a. HelloHello!
- b. Hello!Hello
- c. Hello! Hello
- d. Hello



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def modify_string(text: str) -> str:
    text += "!"
    return text
```

```
original = "Hello"
modify_string(original)
print(original)
```

- a. HelloHello!
- b. Hello!Hello
- c. Hello! Hello
- d. Hello



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА





Обработка элементов списка функцией

Обработка элементов списка функцией

Напишите функцию, которая принимает другую функцию и список произвольных элементов. Примените переданную функцию ко всем элементам списка и верните новый список. Добавьте документацию и аннотации типов для всех параметров и возвращаемого значения.

Данные

```
numbers = [1, 2, 3, 4, 5]
```

Пример вывода

```
[2, 4, 6, 8, 10]
```


Домашнее задание

Объединение данных в строку

Напишите функцию, которая принимает список любых данных (строки, числа, списки, словари) и возвращает их строковое представление, объединённое через " | ". Добавьте документацию и аннотации типов для всех параметров и возвращаемого значения.

Данные:

```
data = [42, "hello", [1, 2, 3], {"a": 1, "b": 2}]
```

Пример вывода:

```
42 | hello | [1, 2, 3] | {'a': 1, 'b': 2}
```

Домашнее задание

Сумма вложенных чисел

Напишите функцию, которая принимает список словарей, где каждый словарь содержит имя пользователя и список баллов. Функция должна вернуть сумму всех чисел. Добавьте документацию и аннотации типов для всех параметров и возвращаемого значения.

Данные:

```
data = [  
    {"name": "Alice", "scores": [10, 20, 30]},  
    {"name": "Bob", "scores": [5, 15, 25]},  
    {"name": "Charlie", "scores": [7, 17, 27]}  
]
```

Пример вывода:

Итоговый балл: 156

Заключение

